

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 4

The xv6 Shell from the Outside

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Reițman Patricia, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. Fix sleep for the current xv6 **2**

3 Part 2. bash versus the xv6 shell **3**
3.1 Comparative table 4
3.2 Observations 4

4 Part 3. Who expands *? A program that prints its argv **5**
4.1 Observations 6

5 Part 4. Read the shell **7**
5.1 Observations 8

6 Conclusion **9**

1 Goal of the work

The purpose of this laboratory work is to study the architecture and behaviour of a Unix command interpreter by comparing the minimalist shell of xv6 (`user/sh.c`) with the GNU `bash` interpreter on Linux. The work covers lexical analysis of arguments, I/O descriptor redirection, pipes, variable evaluation, wildcard expansion (globbing), quote removal, and the execution of built-in versus external commands.

2 Part 1. Fix sleep for the current xv6

In the current versions of xv6 (from August 2025 onwards) the `sleep()` system call was renamed to `pause()`. In this part I checked the declaration in `user/user.h`, updated the source file `user/sleep.c` and confirmed that the program is listed in the `Makefile`.

```
nastea@LinuxLabs:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

The result confirms that the system call is now called `pause`. The updated `user/sleep.c` is:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

After recompiling with `make qemu`, I tested the utility in the xv6 shell:

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

```
25:int pause(int);
```

Figure 1: The grep result in user.h

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

Figure 2: The updated sleep.c

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 3: sleep running in xv6, with and without an argument

3 Part 2. bash versus the xv6 shell

I ran the same 10 commands in both shells: in `bash`, inside the `~/shtest` directory that contains the file `README`, and in the xv6 interpreter.

3.1 Comparative table

Command	Tests	bash (Linux)	xv6 shell
`echo hello > f; cat < f`	redirection	Prints `hello`. Correctly redirects descriptors 1 and 0.	Prints `hello`. Redirects files via `open` and `dup`.
`ls | grep READ`	pipe	Prints `README`. Connects stdout to stdin via `pipe`.	Prints `README`. Complete implementation of anonymous pipes.
`(echo a; echo b) | wc`	grouping, list	Prints `2 2 4` (lines, words, bytes).	Prints `2 2 4`. Evaluates the grouped subshell.
`echo one; echo two`	sequence	Prints `one`, then `two`.	Prints `one`, then `two` (the `;` operator).
`echo bg &`	background	Prints the background PID `[1] 11330` and `bg`.	Prints `bg` asynchronously, without blocking the prompt.
`ls *`	globbing	Expands `*` to the files in the folder (`README f`).	Error: `ls: cannot open *`. Looks literally for a file named `*`.
`echo $HOME`	variables	Replaces the variable with the path `/home/vitalie`.	Prints the literal string `$HOME`. No variable support.
`echo "a b"`	quotes	Prints `a b`. Quotes are removed by the shell.	Prints `"a b"` with the quotes included.
`mkdir d; cd d; ls ..`	PATH	Creates `d`, enters it and runs `ls ..` found via `$PATH`.	`ls ..` fails (`exec ls failed`) because there is no `PATH` variable.
`arrow up, Tab, Ctrl+R`	history, completion	Autocompletes names and navigates history.	Prints raw control characters (no advanced terminal support).

3.2 Observations

1. **Common features versus bash-only features:** Both interpreters support I/O redirection (`<`, `>`), pipes (`|`), sequential lists (`;`), and background execution (`&`) at the system-call level. Features that only `bash` provides are wildcard expansion (globbing `*`), environment variable substitution (`$`), quote interpretation, executable lookup via `PATH`, command history, and automatic completion.

2. **The behaviour of `ls` after `cd d` in xv6:** In xv6, `ls` stops working inside `d` because the interpreter has no `PATH` variable that lists the directories containing binaries. The shell looks for the executable only in the current directory. Since the `ls` binary is in the root directory, the working command from `d` is `../ls ...`

3. **Impact of the missing features:** In daily use the most missed feature is Tab completion together with command history, without which productivity drops considerably. From the point of view of implementation effort, the easiest feature to add to `sh.c` would be removing the quote characters while the parameters are processed.

```
nastea@LinuxLabs:~/shtest$ echo hello > f; cat < f
hello
nastea@LinuxLabs:~/shtest$ ls | grep READ
README
nastea@LinuxLabs:~/shtest$ ls *
README  f
```

Figure 4: The commands run in bash

```
$ echo hello > f; cat < f
hello
$ ls | grep READ
README
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
```

Figure 5: The same commands run in xv6

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
$ ../ls ..
```

Figure 6: ls failing after cd d, and the working alternative

4 Part 3. Who expands *? A program that prints its argv

I wrote the programs `args.c` for xv6 and `args_linux.c` for Linux, to see how command-line parameters arrive in the `argv[]` array.

Results in Linux (`bash`):

```
nastea@LinuxLabs:~/shtest$ ./args one     two     three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

vitalie@LinuxLabs:~/shtest$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]

vitalie@LinuxLabs:~/shtest$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vitalie]

vitalie@LinuxLabs:~/shtest$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Results in xv6:

```
$ args one     two     three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]

$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
```

4.1 Observations

1. **Output of `args *.c "x y"` compared:**
   - In `bash` the program receives 3 arguments (`argc = 3`). `*.c` is expanded to `args_linux.c`, and `"x y"` is one argument `[x y]` with the quotes removed.
   - In xv6 the program receives 4 arguments (`argc = 4`). `*.c` is not expanded and arrives as `[*.c]`. Because xv6 does not interpret quotes, the space between `x` and `y` splits the string into two arguments: `["x]` and `[y"]`.
2. **Who expands `*.c` and removes the quotes:** The command interpreter (the

shell) is responsible for expanding file names and removing quotes. Neither the kernel nor the executable program does this; the program receives the already processed arguments from the shell through the `exec` call. In xv6 the interpreter has no such logic, so all characters are passed through unchanged.

3. **Multiple spaces:** In both shells a sequence of spaces acts as a separator. The shell ignores the extra spaces and produces clean, individual arguments.

```
        $U/_args\
```

Figure 7: The `$U/_args` line added to UPROGS in the Makefile

```
$ args one     two     three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

Figure 8: args output in xv6

```
nastea@LinuxLabs:~/shtest$ ./args one     two     three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

Figure 9: args_linux output in bash

5 Part 4. Read the shell

I analysed the source file `user/sh.c` (499 lines) with `wc` and `grep`:

```
nastea@LinuxLabs:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c

nastea@LinuxLabs:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10

nastea@LinuxLabs:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
```

```
93:  case LIST:
101:  case PIPE:
125:  case BACK:
...
nastea@LinuxLabs:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

5.1 Observations

1. **The five command types and the characters that produce them:**
   - `EXEC` (1) – a simple command, the words separated by spaces.
   - `REDIR` (2) – I/O redirection, produced by `<` and `>`.
   - `PIPE` (3) – an anonymous pipe between processes, produced by `|`.
   - `LIST` (4) – a sequence of independent commands, produced by `;`.
   - `BACK` (5) – background execution, produced by `&`.
2. **Where `cd` is handled and why it is not a separate program:** `cd` is handled directly in the main loop of `main()` (line 170 of `user/sh.c`), which calls the `chdir(cmd + 3)` system call itself. It cannot be an external program, because running an external program requires `fork()` to create a new process. A directory change made in a child process would only affect that child, and the parent shell would stay in the old directory.
3. **Searching for `*` and `$` in `sh.c`:** No line of `sh.c` checks or processes the `*` or `$` characters. This explains the results of Parts 2 and 3: the xv6 shell passes these characters unchanged into the program's parameter list.

```
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
```

Figure 10: The command kinds defined in sh.c

```
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
```

Figure 11: Where each kind is run

```
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 12: The code that handles cd

6 Conclusion

In this laboratory work I studied the architecture of the Unix command interpreter by comparing the minimalist xv6 shell with GNU `bash`. The experiments showed a clear division of responsibilities between the kernel and the interpreter: the kernel provides the primitives (`fork`, `exec`, `pipe`, `chdir`), while the shell parses the command line. Wildcard expansion (`*`), variable substitution (`$`) and quote removal are done by the interpreter before the parameters reach the programs. Reading `sh.c` explained why `cd` must be built into the shell, which illustrates the basic process-management principles of Unix.